

35 Racing

Documentación del proceso de desarrollo

Proyecto final de Programación sobre Redes

Videojuego de carreras 2D multijugador en red local (Java 17 + libGDX)

Repositorio: https://github.com/joakinkong/35-Racing

Documento actualizado hasta la etapa 10 · generado el 05/11/2026 · versión 17441e5

Integrantes
Joaquin Barreira
Madai Andacaba
Antonella Vivacqua
Joaquin Rodrigues
Mateo Guinsburg

Contenido

Cómo trabajamos y línea de tiempo

Etapa 0: Preparación

Etapa 1: Pre-entrega 1

Etapa 2: Esqueleto y pantallas

Etapa 3: Auto y circuito (1 jugador local)

Etapa 4: Reglas de carrera y dos autos locales

Etapa 5: Diseño del protocolo de red

Etapa 6: Conexión y lobby por TCP

Etapa 7: Carrera sincronizada por UDP

Etapa 8: Robustez de red

Etapa 9: Pruebas en LAN real

Etapa 10: Circuitos vectoriales y HUD estilo Fórmula 1

Cómo fuimos trabajando: proceso de desarrollo de 35 Racing

Este documento muestra el proceso real de trabajo del proyecto, etapa por etapa: qué se pidió, qué se hizo, qué decisiones se tomaron, qué problemas aparecieron y cómo se comprobó que funcionaba. Está escrito para que se pueda seguir en orden y verificar contra el historial de Git del repositorio.

Cómo trabajamos

Uso de inteligencia artificial. El proyecto se desarrolla con Claude Code (un asistente de IA que trabaja dentro del repositorio) siguiendo una guía de prompts por etapas. La IA escribe y revisa código y documentación; el grupo decide qué se hace, aprueba los planes, prueba el juego y pide cambios. Los commits se hacen desde la cuenta de Joaquín Barreira y no llevan la línea `Co-Authored-By`, por lo que la IA no figura como colaboradora en GitHub: su uso se declara acá.

El ciclo que se repitió en cada etapa fue este:

1. **Plan primero.** Antes de escribir código, el asistente muestra un plan (clases a crear o modificar y decisiones a tomar) y se espera la confirmación del grupo. Es una regla del archivo CLAUDE.md.
2. **Rama de trabajo.** Cada etapa se hace en una rama `feature/...` que sale de `develop`.
3. **Implementación.** Código en español, con constantes en `Config` y `ParametrosAuto` (sin números mágicos) y el modelo del juego separado de lo gráfico.
4. **Verificación.** Además de compilar con Gradle, se escribieron pruebas descartables que abren el juego con la ventana oculta, simulan autos con entradas guionadas y sacan capturas de pantalla. Los resultados numéricos de cada prueba están en los documentos de cada etapa.
5. **Corrección.** Los errores que encontró la prueba o el grupo al jugar se corrigieron en commits propios, con prefijo `Fix:`.
6. **Integración.** Pull request de la rama de trabajo a `develop`, y de `develop` a `main` al cerrar un hito. El CHANGELOG se actualiza en cada etapa.

Los commits usan los prefijos `Feat:`, `Fix:`, `Docs:` y `Refactor:`, como pide la guía.

Línea de tiempo

Las etapas 0 a 4 se hicieron el 28/09/2026, entre las 21:16 y las 23:45 (hora local, según los commits). Esta documentación del proceso se armó a continuación, y las etapas 5 a 9 el 29/09/2026. La etapa 10 se hizo el 05/11/2026.

Etapa	Qué quedó funcionando	Pull request
0. Preparación	Proyecto LibGDX (core + lwjgl3) con Java 17, repositorio público en GitHub con ramas `main` y `develop`	(commits directos)

Etapa	Qué quedó funcionando	Pull request
1. Pre-entrega 1	README, CHANGELOG, `.gitignore` y la propuesta pasada a Markdown para la Wiki	#1
2. Esqueleto y pantallas	Menú, unirse, lobby, carrera y resultados navegables, con estética de Fórmula 1	#2
3. Auto y circuito	Un auto manejable en un circuito, con física arcade, cámara y HUD	#3
4. Reglas de carrera	Carrera completa para 2 jugadores en el mismo teclado: cuenta regresiva, vueltas, posiciones, choques y resultados	#4 y #5
Documentación del proceso	Esta documentación por etapas y su versión en PDF	#6
5. Protocolo de red	Especificación completa de la comunicación en red en docs/PROTOCOLO.md (diseño, todavía sin código)	#7
6. Conexión y lobby por TCP	Crear una partida, unirse por IP, lobby en vivo con elección de auto y "listo", e inicio de la carrera (cada uno todavía corre la suya)	#9
7. Carrera sincronizada por UDP	La carrera se juega en red: el servidor la simula, los clientes mandan sus teclas y dibujan lo que llega; resultados, revancha y tolerancia a pérdida de paquetes	#9 y #10
8. Robustez de red	Aviso y vuelta al menú si dejan de llegar datos UDP, fallas del servidor que cierran la partida con motivo, y cierre de ventana limpio desde cualquier pantalla	#12
9. Pruebas en LAN real	Carrera completa con revancha entre 2 notebooks en el hotspot de un celular, y pérdida de paquetes simulada. En la red del colegio no se pudo unir nadie (firewall sin permisos de administrador)	Solo documentación
10. Circuitos vectoriales y HUD	Los 24 circuitos de 2026 como geometría (sin tiles ni pixel art), dibujados con antialiasing; física más exigente; elección de circuito en la prueba local; torre de tiempos estilo Fórmula 1	Todavía sin abrir (rama `feature/estetica-f1`)

Estado del repositorio al cerrar la etapa 9: `main` tiene todo hasta la etapa 8; la documentación de la etapa 9 está en la rama `docs/pruebas-lan`.

Lo que sigue

- **Antes de seguir:** que los cinco integrantes lean docs/PROTOCOLO.md, porque es lo que tienen que poder explicar.
- **Pendiente de la etapa 9:** probar con 3 y con 5 jugadores, y conseguir que abran los puertos TCP 7777 y UDP 7778 en las PC del colegio (hace falta un administrador). Mientras tanto, la demo en red se puede hacer con el hotspot de un celular.

- **Pendiente de arte:** los sprites. Los circuitos ya son definitivos desde la etapa 10, pero los autos siguen siendo los provisorios (ver etapa 3).
- **Pendiente de la etapa 10:** los pianos en las curvas, el puente de Suzuka y repetir la prueba en LAN con los circuitos nuevos.

Participación del equipo

Hasta la etapa 9 todos los commits salieron de la cuenta de Joaquín Barreira. El resto del grupo tiene invitación como colaborador del repositorio, y las tareas de arte (circuito y sprites) están asignadas al resto del equipo según la guía. Cuando haya commits de los demás integrantes, se van a reflejar en el historial y en esta documentación.

Etapa 0: Preparación

Objetivo

Dejar listo el entorno y el proyecto base, según la guía: herramientas instaladas, proyecto generado con gdx-liftoff, importado en Eclipse, documentos de la propuesta dentro del repositorio y repositorio público en GitHub.

Qué se hizo

Herramientas. Se verificó lo que ya estaba instalado: JDK 21 (cumple el mínimo de Java 17), Git 2.53 con el nombre y el mail configurados, y la CLI de GitHub (`gh`), que después se usó para crear ramas y gestionar colaboradores.

Proyecto con gdx-liftoff (versión 1.14.2.3), con estas opciones:

Opción	Valor
Nombre del proyecto	35 Racing
Paquete	`ar.edu.et35.racing`
Clase principal	`Main`
Plataformas	Core y Desktop (LWJGL3), nada más
Extensiones	Freetype únicamente (sin Box2D ni Ashley)
Plantilla	Classic

El proyecto se importó en Eclipse como *Existing Gradle Project* y `Lwjgl3Launcher` abrió la ventana con el logo de libGDX. Antes de importarlo se comprobó que compilara con Gradle (`gradlew lwjgl3:classes` , sin errores).

Documentos del repositorio.

- `docs/propuesta/propuesta.pdf` : la propuesta aprobada (versión 2), con un nombre sin espacios para poder referenciarla.
- `docs/propuesta/img/` : las tres figuras de la propuesta.
- `CLAUDE.md` : el contexto fijo del proyecto para el asistente (stack, arquitectura, convenciones y forma de trabajo), con el nombre y el paquete ya completados.

Repositorio. joakinkong/35-Racing, público (necesario para tener Wiki) y con la Wiki activada. Ramas `main` y `develop` . Se invitó como colaboradores a `jasinski1988` (profesor), `JoacoR2007` , `mateoguinsburg35` y `antonellavivacqua` .

Decisiones

- **Paquete `ar.edu.et35.racing`:** sigue la convención de dominio invertido e identifica a la escuela.
- **Repositorio `35-Racing`:** GitHub no acepta espacios en el nombre. Cuando se fue a crear ya existía uno vacío con ese nombre en la misma cuenta, y se usó ese en lugar de crear otro.
- **Java 17 en Gradle:** gdx-liftoff genera el proyecto compilando para Java 8, pero las pautas piden Java 17. Se cambió `sourceCompatibility`, `targetCompatibility` y `release` a 17 en `build.gradle` y `lwjgl3/build.gradle`, y se volvió a compilar. Con Java 8 no se habrían podido usar funciones del lenguaje como los `record`, que el proyecto usa más adelante.

Problemas encontrados

- `gh` no tenía sesión iniciada: se resolvió con `gh auth login`.
- El repositorio ya existía vacío: se verificó que estaba vacío, era público y tenía la Wiki activa antes de usarlo.

Commits

- `9b1bca7`: Configuración inicial del proyecto con LibGDX Liftoff.
- `ecaae86`: Fix: Compilar con Java 17 según las pautas de la materia.

Etapa 1: Pre-entrega 1

Objetivo

Completar la documentación mínima de la primera pre-entrega: propuesta en la Wiki, `.gitignore`, CHANGELOG y README.

Qué se hizo

- **Propuesta en Markdown para la Wiki.** Se extrajo el texto del PDF de la propuesta y se pasó a `docs/wiki/Propuesta-del-Proyecto.md`, respetando el texto y la numeración de secciones. La portada quedó resumida en un encabezado con el título, la fecha y los integrantes. Se dejaron afuera la tabla de contenidos y la sección de aprobación, y se insertaron las tres figuras con su epígrafe. Después la página se cargó a mano en la Wiki de GitHub.
- `.gitignore`**.** Se revisó el que generó liftoff contra la lista pedida: ya cubría todo (Gradle, `bin/`, `.project`, `.classpath`, `.settings/`, `.idea/`, `Thumbs.db`, etc.) salvo `.claude/settings.local.json`, que se agregó.
- **CHANGELOG.** Se creó `CHANGELOG.md` en español con formato Keep a Changelog y una entrada `[0.1.0]` para el proyecto generado, el repositorio, el README, el CHANGELOG y la Wiki.
- **README.** Se reemplazó el README genérico de liftoff por uno propio: nombre, integrantes, descripción breve basada en la propuesta, tecnologías, enlace a la Wiki, cómo compilar y ejecutar (Eclipse y consola) y estado actual.

Decisiones

- **Fecha del CHANGELOG en formato ISO** (`2026-09-28`), que es el que usa Keep a Changelog.
- **README solo con lo que ya existe:** describe el estado real ("configuración inicial y estructura del proyecto") y no promete funciones que todavía no están.

Problemas encontrados

- **Lectura del PDF.** La herramienta de lectura de PDF del asistente necesita `poppler`, que no está instalado. Se resolvió extrayendo el texto con la biblioteca `pypdf` de Python, que además hizo falta configurar con codificación UTF-8 para que no fallaran los caracteres especiales.
- **Link roto en el CHANGELOG.** El CHANGELOG enlazaba al tag `v0.1.0`, que nunca se creó, así que daba error 404. Se quitó el enlace en lugar de crear un tag que nadie había pedido (`6436374`).

Checklist de la pre-entrega 1 (estado al cerrar la etapa)

Ítem	Estado
El proyecto compila y corre en escritorio	Cumplido

Ítem	Estado
`.gitignore` adecuado	Cumplido
README con nombre, integrantes, descripción, tecnologías, Wiki e instrucciones	Cumplido
CHANGELOG con la entrada inicial	Cumplido
Wiki activada, con la propuesta	Cumplido
Todos los integrantes y los 3 profesores como colaboradores	Incompleto: había 4 invitaciones enviadas y faltaban Madai y dos de los profesores
Commits de varios integrantes	Incompleto: todos los commits eran de una sola cuenta

Commits y pull request

- `85c0daf` : Docs: README, CHANGELOG y .gitignore para pre-entrega 1.
- `a023fda` : Docs: Propuesta del proyecto en Markdown para la Wiki.
- `6436374` : Docs: Quitar link roto al tag v0.1.0 del CHANGELOG.
- Pull request #1: `develop` a `main` .

Etapa 2: Esqueleto y pantallas

Objetivo

Armar el esqueleto del juego, todavía sin física ni red: cinco pantallas navegables con botones (menú principal, unirse, lobby, carrera y resultados), con ESC para volver, una interfaz hecha con Scene2D y una resolución virtual de 640×360 con estilo pixel art. Además, el grupo pidió que todo el juego tenga una **estética de Fórmula 1**.

El plan (se mostró y se confirmó antes de programar)

Clase	Qué hace
`util/Config`	Constantes globales (`MAX_JUGADORES = 5` , `VUELTAS = 3` , `TICKS_POR_SEGUNDO = 60` , puertos 7777 y 7778, resolución virtual, ventana inicial)
`util/Paleta`	Colores del estilo visual
`util/Recursos`	Punto único que crea y libera el `AssetManager` , el `SpriteBatch` , las fuentes y el `Skin`
`pantallas/PantallaBase`	Clase base con la escena, el `FitViewport` de 640×360, el manejo de ESC y el marco visual
`MenuPrincipal` , `UnirsePartida` , `Lobby` , `Carrera` , `Resultados`	Las cinco pantallas
`Main`	Pasa a extender `Game` y decide qué pantalla se muestra

También se crearon los paquetes `juego` , `vista` , `red` , `pantallas` y `util` , cada uno con un `package-info.java` de una línea que dice qué va ahí.

Qué se hizo

- **Interfaz sin archivos externos.** El `Skin` se arma por código: la fuente por defecto de libGDX y rectángulos de color como fondos. Para que el pixel art no se vea borroso al escalar se usa filtro *Nearest*.
- **Resolución virtual 640×360** con `FitViewport` . La ventana abre en 1280×720, es redimensionable y no baja de 640×360; al agrandarla, el contenido escala con barras a los costados en lugar de deformarse.
- **Estética de Fórmula 1:** fondo color carbono (`#15151E`), rojo de largada (`#E10600`) y blanco; una bandera de cuadros arriba y abajo de cada pantalla; cinco luces rojas de largada bajo el título del menú; una grilla de largada P1 a P5 en el lobby; y pianos rojo y blanco en el fondo de la carrera.
- **Navegación completa** entre las cinco pantallas, con ESC para volver al menú.

Decisiones

- `PantallaBase` **compartida.** Las cinco pantallas repiten la misma estructura (escena, ESC, marco). Se resolvió una sola vez en una clase base.
- **Un solo** `SpriteBatch`**.** Las escenas comparten el de `Recursos` en lugar de crear uno por pantalla, y así se libera en un único lugar.
- **Cambio de pantalla diferido.** `Main.irA(...)` cambia de pantalla con `Gdx.app.postRunnable`. Se llama desde un botón de la pantalla vieja, y destruirla en medio de su propio evento puede fallar.
- **Sin acentos que no existan en la fuente.** Antes de escribir los textos se verificó que la fuente por defecto trae ó, é, ñ, ¡ y Ó, pero no las flechas; por eso no se usan flechas en la interfaz.
- **Botón "Finalizar carrera" provisorio** para poder llegar a Resultados sin lógica de carrera. Se quitó en la etapa 4.

Verificación

- Compila con Gradle sin errores.
- El juego arrancó y siguió corriendo 25 segundos sin excepciones (se detuvo desde afuera).
- Se sacaron capturas de las pantallas con una ventana oculta.

Capturas

Menú principal	Unirse
	

Lobby	Resultados (datos de ejemplo)
	

Commits y pull request

- `c007aca` : Feat: Config, Paleta y Recursos con Skin por código y estilo F1.
- `76eebb3` : Feat: Esqueleto de pantallas con navegación y ventana 1280x720.
- Pull request #2.

Una lección de flujo de trabajo

El pull request #2 se hizo directamente contra `main`, en lugar de contra `develop`, y así se salteó la rama de integración. No se rompió nada, pero después se corrigió la práctica: los pull requests de cada etapa van a `develop`, y `develop` pasa a `main` al cerrar un hito (ver etapas siguientes).

Etapa 3: Auto y circuito (1 jugador local)

Objetivo

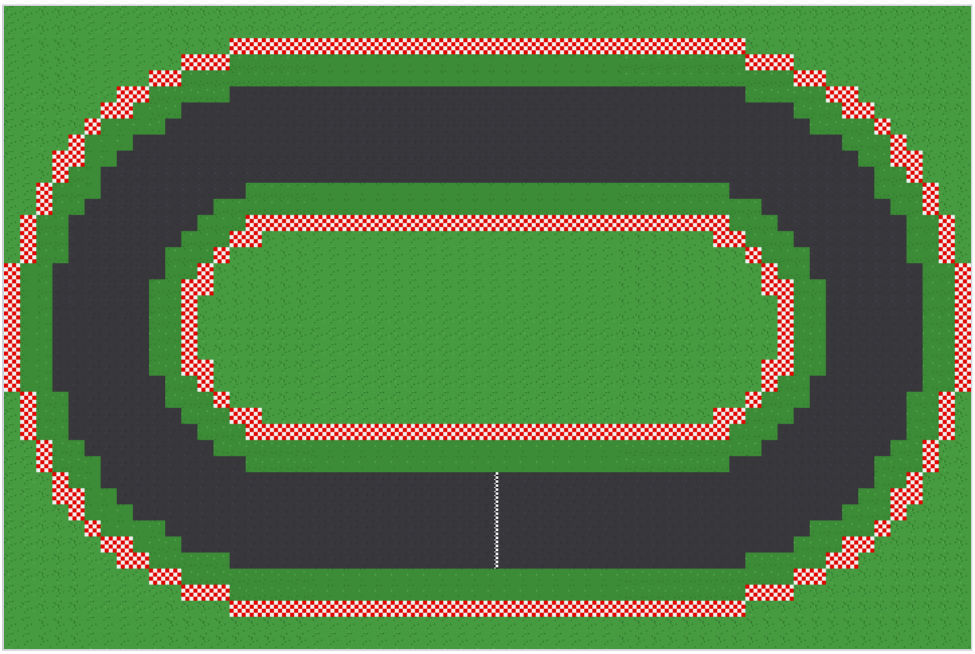
Un auto manejable en un circuito, con un solo jugador local y sin red: física arcade propia (sin Box2D), circuito cargado desde un mapa de Tiled, cámara que sigue al auto y simulación a paso fijo desacoplada del dibujo.

El circuito: un requisito que faltaba

La guía indica que la persona de arte arma a mano `assets/circuitos/circuito1.tmx` en Tiled, con tiles de 32×32 y propiedades personalizadas. Ese archivo todavía no existía, y sin él no se podía probar nada. Se propuso y se aprobó generar un **circuito provisorio** con un script, con el mismo formato exacto que pide la guía, para poder seguir sin bloquearse:

- Un óvalo de 60×40 tiles con pista de 6 tiles de ancho, pasto de escape de 2 tiles a cada lado y muros con pianos rojo y blanco.
- Un tileset de 5 tiles en pixel art (pasto, pista, muro, línea de meta y una variante de pasto) generado por código.
- Capa de tiles `suelo` con la propiedad `superficie` (`pista`, `pasto` o `muro`) en cada tile, capa de objetos `checkpoints` (10 rectángulos con `orden`, donde el 0 es la meta), capa `largada` (5 lugares con `posicion`) y la propiedad `anguloLargada` en el mapa.

Cuando arte tenga el circuito definitivo, lo reemplaza sin tocar código mientras respete esas capas y propiedades.



Vista general del circuito provisorio

Qué se hizo

Modelo (`juego`), sin ninguna clase gráfica porque después lo usa el servidor:

Clase	Qué hace
`Circuito`	Datos puros: grilla de superficies, checkpoints ordenados, lugares y ángulo de largada; detecta si un círculo choca con un muro
`CargadorCircuito`	Convierte el `TiledMap` en un `Circuito` y valida que el mapa tenga todas las capas y propiedades
`Superficie`	`PISTA` , `PASTO` o `MURO`
`EntradaAuto`	Lo que hace el jugador: acelerar, frenar y girar (-1, 0 o 1)
`ParametrosAuto`	Todos los valores de la física en un solo lugar
`Auto`	Posición, ángulo y velocidad, con la física arcade

Física del auto. La velocidad se separa en una componente hacia adelante y otra lateral respecto de adonde mira el auto, y la lateral se va perdiendo de a poco: de ahí sale el derrape leve. Tiene aceleración, freno, reversa lenta, velocidad máxima y rozamiento; el giro depende de la velocidad (parado no gira, en reversa gira al revés); en el pasto baja la velocidad máxima y sube el rozamiento; y contra un muro rebota y pierde velocidad sin atravesarlo. La colisión se resuelve un eje por vez, con el auto como círculo.

Vista (`vista`). `VistaCircuito` dibuja el mapa, `VistaAuto` dibuja el auto como un monoplaza armado con rectángulos (hasta que haya sprites) y `CamaraSeguimiento` sigue al auto con un leve adelanto hacia donde va.

Pantalla `Carrera` . Simulación a paso fijo de 1/60 s con un acumulador, y dibujo que interpola entre el último tick y el siguiente para que el movimiento sea fluido aunque la frecuencia de cuadros varíe. Se maneja con flechas o WASD.

Decisiones

- **Física propia y sin Box2D:** lo exigen las pautas.
- **Modelo separado de lo gráfico:** `Circuito` , `Auto` y `EntradaAuto` no usan nada de dibujo ni de teclado, para que el servidor pueda simular la carrera.
- **Colisión por eje:** si el auto chocaría al moverse en un eje, ese eje no se mueve y su velocidad rebota con pérdida. Así no atraviesa muros ni se traba.
- **Tope de tiempo por cuadro (0,25 s):** un tirón, por ejemplo al mover la ventana, no dispara cientos de ticks de golpe.

Problemas encontrados y cómo se resolvieron

1. **En el pasto con el acelerador apretado, el auto se frenaba por completo.** Se detectó al revisar el código antes de correrlo: el rozamiento del pasto (390 px/s^2) le ganaba a la aceleración (220 px/s^2). Se

cambió para que en el pasto con el acelerador la velocidad la limite el máximo permitido y el auto avance despacio.

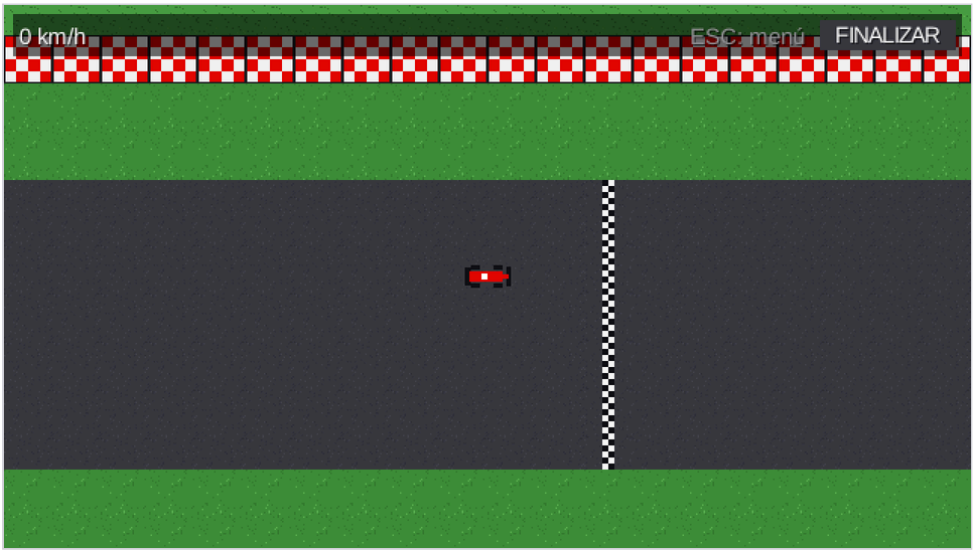
2. **Marcha atrás imposible en el pasto.** Lo encontró la prueba automática: un auto pegado a un muro, en la zona de pasto, no podía salir marcha atrás, porque el rozamiento por tick (5 px/s) era mayor que la aceleración en reversa por tick (2 px/s). Se corrigió para que el rozamiento solo actúe cuando no se toca ningún pedal.

Verificación

Se escribió una prueba descartable (no forma parte del repositorio) que abre el juego con la ventana oculta, carga el circuito real y simula autos con entradas guionadas. Resultados:

Prueba	Resultado
El `.tmx` carga	1920×1280 px, 10 checkpoints, 5 lugares de largada, todos sobre pista y sin tocar muros
Acelerar 3 s en recta	Llega a la velocidad máxima de 360 px/s
Parado con el giro apretado	El ángulo no cambia
Frenar parado	Retrocede, con tope de 90 px/s
Pasto acelerando	162 px/s, que es el 45 % del máximo, como se configuró
Chocar de frente 8 s contra un muro	0 veces dentro del muro; después sale marcha atrás (antes del arreglo, se quedaba trabado)
Piloto automático de 2 vueltas	Completa las 2 vueltas (11,7 s y 10,7 s) sin tocar un muro

También se sacó una captura del auto en la largada.



Auto en la línea de largada

Cómo ajustar la sensación de manejo

Solo se toca `ParametrosAuto` : `ACELERACION` y `VELOCIDAD_MAX` (arranque), `GIRO_MAX` y `VELOCIDAD_GIRO_COMPLETO` (giro), `AGARRE` (más alto, menos derrape), `ROZAMIENTO` (cuánto tarda en frenarse solo), `FACTOR_VELOCIDAD_PASTO` y `ROZAMIENTO_PASTO` (pasto), `REBOTE` y `PERDIDA_POR_ROCE` (muros).

Commits y pull request

- `9df7882` : Feat: Circuito provisorio circuito1.tmx con tileset en pixel art.
- `5fb156e` : Feat: Modelo del auto y del circuito con física arcade.
- `80d0f95` : Feat: Carrera con un auto manejable, cámara y HUD.
- Pull request #3, esta vez sí hacia `develop` .

Etapa 4: Reglas de carrera y dos autos locales

Objetivo

Sumar las reglas de una carrera completa, todavía sin red: cuenta regresiva, vueltas validadas por checkpoints, posiciones, choques entre autos, fin de carrera y resultados. Es el paso de la propuesta en que "se incorporará el segundo vehículo controlado localmente para probar la lógica de colisiones y vueltas".

El plan (se mostró y se confirmó antes de programar)

Además de las reglas, el plan proponía tres decisiones que el grupo confirmó: renombrar la pantalla `Carrera` a `PantallaCarrera` (para no chocar con el modelo `juego.Carrera`), que "Prueba local" sea de 2 jugadores en el mismo teclado (J1 con WASD y J2 con flechas) y usar los nombres provisorios "Jugador 1" y "Jugador 2".

Qué se hizo

Modelo (`juego`), sin clases gráficas. `Carrera` recibe las entradas de cada auto por identificador (`Map<Integer, EntradaAuto>`) y avanza un paso de simulación. Es la clase que más adelante va a correr en el servidor, así que no lee el teclado. Sus reglas:

- **Estados:** `CUENTA_REGRESIVA`, `EN_CURSO` y `TERMINADA`. La cuenta dura 3 segundos y hasta el "¡YA!" las entradas se ignoran y los autos no se mueven.
- **Vueltas:** el auto sale detrás de la meta y tiene que pasar por los checkpoints en orden y cruzar la meta. Se mide el tiempo de cada vuelta, la mejor vuelta y el tiempo total.
- **Posiciones:** primero los que terminaron, por tiempo; después, por vueltas completadas, por checkpoint alcanzado y por distancia al próximo checkpoint.
- **Fin:** cuando el primero completa las 3 vueltas, el resto tiene 30 segundos para terminar. Pasado ese tiempo, o si todos terminaron antes, la carrera pasa a `TERMINADA`.
- **Choques entre autos:** dos círculos de igual masa que se separan por el solapamiento (si un muro le impide moverse a uno, el otro absorbe lo que falta) y se reparten la velocidad con un rebote parcial.

Clases nuevas: `Carrera`, `EstadoCarrera`, `Participante` y `ResultadoJugador`.

Pantalla y vista. `PantallaCarrera` maneja 1 o 2 jugadores locales, con un panel de HUD por jugador (vuelta, posición, tiempo actual y mejor vuelta, con una franja del color de su auto), un cartel grande para la cuenta regresiva y el "¡YA!", y el pase automático a Resultados 2,5 segundos después de terminar. `Resultados` muestra datos reales. Se agregaron `Tiempo` (formato `m:ss.mmm`) y una paleta con un color por auto.

Decisiones

- `Carrera` **pensada para el servidor:** sin teclado, sin dibujo y con las entradas por identificador, porque va a ser lo que simule el servidor autoritativo cuando llegue la red.

- **Un auto que terminó deja de recibir entradas** y se va frenando solo.
- **Los que no terminan** quedan ordenados por avance y su tiempo total se muestra con guiones.

Problemas encontrados y cómo se resolvieron

La verificación y el uso real encontraron varios problemas. Estos son los más importantes, en el orden en que aparecieron:

1. **Manejar en contramano avanzaba checkpoints** (encontrado revisando la regla). Con la condición "cruzar el checkpoint hacia adelante", un auto que recorre la pista al revés igualmente sumaba un checkpoint por vuelta. Se agregó una segunda condición: además de ir hacia adelante, el auto tiene que avanzar en el sentido de la pista en ese checkpoint (calculado desde la geometría del circuito).
2. **La primera prueba de contramano no probaba nada.** Reportó 0 checkpoints cruzados, porque el piloto automático de prueba no podía dar la vuelta estando parado (un auto parado no gira). Se corrigió la prueba, y con la versión correcta cruzó más de 80 checkpoints al revés sin sumar una sola vuelta.
3. **Los checkpoints solo cubrían la pista** (observación del grupo al jugar). Un auto que se salía al pasto de escape se los salteaba y no sumaba vueltas, aunque ir por el pasto ya es una desventaja porque el auto va más lento. Se regeneró el circuito con checkpoints de muro a muro. Prueba: un piloto que fue el 72 % del tiempo por el pasto completó 2 vueltas en 46,6 s, contra unos 21 s por la pista. Cuentan, pero siguen siendo más lentas.
4. **La cámara perdía de vista al segundo jugador** (observación del grupo al jugar). Con una cámara compartida, cuando los autos se separaban mucho la cámara seguía solo al que iba primero y el otro quedaba fuera de pantalla. Se probaron dos soluciones, y la segunda la pidió el grupo:
   - Primero, pantalla dividida solo cuando los autos no entraban juntos.
   - Después, por pedido del grupo, algo más simple: cada jugador tiene su propia cámara con zoom fijo de 2× y la pantalla queda siempre dividida, sin unirse ni separarse. Se eliminó bastante código: el commit quita 134 líneas y suma 38.
5. **Una captura rara que no era un error del juego.** Al probar la pantalla dividida, un auto aparecía descentrado. La causa era la propia prueba, que movía el auto de lugar durante la cuenta regresiva, cuando el auto no se actualiza; se repitió la prueba moviéndolo con la carrera en curso y quedó centrado. Sirvió para separar un artefacto de la prueba de un error real.
6. **El texto de ayuda del HUD** se perdía sobre los pianos del mapa: se le agregó un fondo, como al resto del HUD.

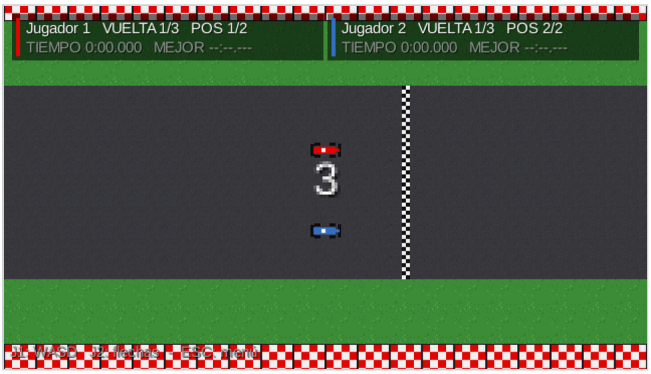
Verificación

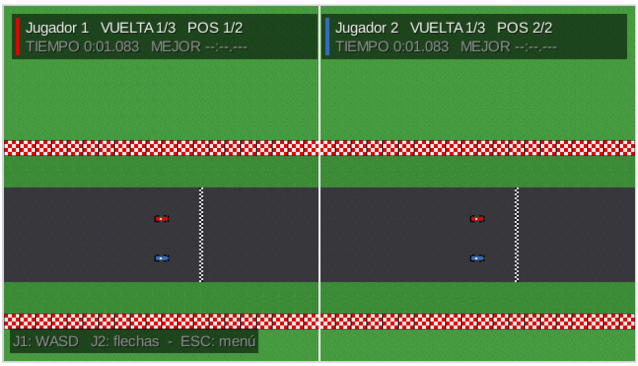
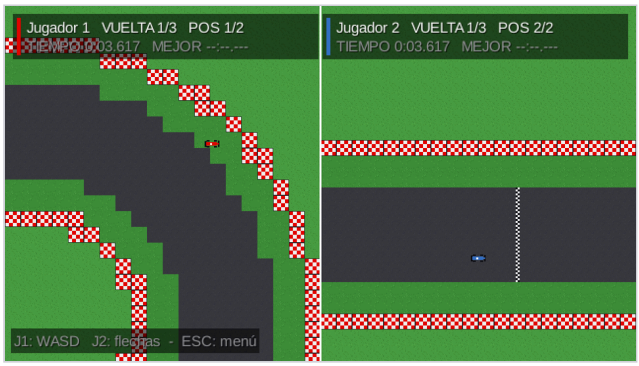
Con la misma técnica de la etapa 3 (prueba descartable con ventana oculta), sobre el circuito real:

Prueba	Resultado
Cuenta regresiva con el acelerador apretado	Dura 3,02 s y el auto se movió 0 px
Sumar un auto con la carrera empezada	Se rechaza
Dos pilotos automáticos, 3 vueltas	Terminan en 32,866 s y 34,100 s, en el orden correcto

Prueba	Resultado
Contramano durante 90 s	83 checkpoints cruzados al revés, 0 vueltas
Choque por atrás	Distancia mínima entre autos de 20 (igual a 2 radios: no se solapan ni se traban)
Cierre de la carrera	30,02 s después de que termina el primero (se pidieron 30)
El que no terminó	Aparece en resultados sin tiempo total
Vueltas por el pasto	2 vueltas completas, 46,6 s

Capturas

Cuenta regresiva	"¡YA!"
	

Pantalla dividida, autos juntos	Pantalla dividida, autos lejos
	



Resultados con datos de la prueba con dos pilotos automáticos

Los nombres "Rapido" y "Lento" de la última captura son de la prueba con pilotos automáticos; en el juego los autos se llaman "Jugador 1" y "Jugador 2".

Commits y pull requests

- `a547b09` : Feat: Reglas de carrera en el modelo (cuenta, vueltas, posiciones y choques).
- `3565910` : Feat: Prueba local a 2 jugadores con HUD, cámara y resultados reales.
- `d05a5b0` : Fix: Checkpoints de muro a muro para que cuenten también por el pasto.
- `71deaca` : Fix: Pantalla dividida en la prueba local cuando los autos no entran juntos.
- `23a261b` : Fix: Zoom fijo en 2x en pantalla dividida para que el corte no cambie la escala.
- `4f385f1` : Refactor: Una cámara por jugador con zoom fijo 2x, sin unir ni separar pantallas.
- `c0b9567` : Docs: Línea en blanco antes de la entrada 0.1.0 del CHANGELOG.
- Pull request #4: rama de trabajo a `develop` .
- Pull request #5: `develop` a `main` , con todo lo de las etapas 3 y 4.

Etapa 5: Diseño del protocolo de red

Objetivo

Diseñar, sin escribir código, cómo se comunican las computadoras durante una partida. El resultado es docs/PROTOCOLO.md: la especificación que se va a implementar en las etapas 6 a 8 y lo que el grupo tiene que poder explicar. La guía pedía cubrir la arquitectura, la máquina de estados de la partida, los mensajes TCP y UDP con su formato exacto, las frecuencias con la cuenta del ancho de banda, la interpolación en el cliente, los hilos, las fallas, la justificación de TCP y UDP y un diagrama de secuencia de una partida completa.

Para esta etapa se usó el modelo Claude Opus 5.5, como recomienda la guía para los pasos de diseño.

El plan (se mostró y se confirmó antes de escribir)

Antes de redactar el documento se presentaron las decisiones principales, y el grupo confirmó estas tres, que eran las que tenían alternativas:

Decisión	Lo elegido	Alternativa que se descartó
Frecuencia de los estados y retraso de dibujo	30 estados por segundo y 100 ms de retraso de interpolación	60 por segundo y 50 ms (menos demora, el doble de tráfico)
Predicción del auto propio	No, en la primera versión	Predecir desde el principio (más complejo)
Detección de caídas	`PING` cada 2 s y conexión perdida a los 6 s sin datos	(sin alternativa planteada)

Qué se decidió

- **Servidor autoritativo dentro del proceso del host,** y el host se conecta a su propio servidor por `127.0.0.1` como un cliente más, así hay un solo camino de código.
- **Un solo hilo toca el estado de la partida** (el de simulación). Los demás hilos le pasan datos por una cola de eventos (lo que llega por TCP y no se puede perder) o por un valor "última entrada" que se sobrescribe (lo que llega por UDP). Así el modelo no necesita sincronización.
- **TCP en texto** (una línea por mensaje, campos separados por `;`) para el lobby, la largada, los resultados y las desconexiones, con una tabla de 8 códigos de error.
- **UDP en binario** con dos paquetes: `ENTRADA` (11 bytes, cliente al servidor, 60 por segundo) y `ESTADO` (132 bytes con los 5 autos, servidor a todos, 30 por segundo). Cada uno lleva un número de secuencia y se descarta todo lo que llega con un número menor o igual al último.
- **El servidor aprende la dirección UDP de cada cliente** con un token que entrega por TCP en la bienvenida: guarda la dirección de origen de la primera `ENTRADA` válida.

Números que se verificaron

Los tamaños de los paquetes se comprobaron armándolos campo por campo con un script (la suma de los tamaños de cada campo), y las cuentas de ancho de banda y demora, calculándolas en el mismo script en lugar de estimarlas a mano:

Qué	Valor
`ENTRADA`	11 bytes (39 con las cabeceras IP y UDP)
`ESTADO` con 5 autos	132 bytes (160 con cabeceras); entra en un paquete sin fragmentarse (el máximo es 1.472)
Tráfico en la placa de red del host con 5 jugadores	28.560 bytes/s, unos 0,23 Mbit/s (el 0,23 % de una red de 100 Mbit/s)
Demora desde la tecla hasta ver el auto propio	unos 135 ms típica, 171 ms en el peor caso
Lo mismo con 60 estados por segundo y 50 ms	unos 77 ms típica, 104 ms en el peor caso

Problemas encontrados y cómo se resolvieron

1. **Los números del plan no coincidían con el diseño final.** En el plan se habían estimado 10 bytes para `ENTRADA` y 141 para `ESTADO`. Al armar los formatos campo por campo quedaron en 11 y 132 bytes (se agregó un byte de tipo a la entrada y se sacó la velocidad del estado, que el cliente puede estimar). El documento usa los números verificados y se le informó al grupo la diferencia. El cambio en el ancho de banda es mínimo.
2. **La demora de 30 Hz con 100 ms resultó mayor de lo que parecía.** El cálculo paso a paso mostró que casi toda la demora es el retraso de interpolación y no la red (en una LAN un paquete tarda de 1 a 5 ms en ir y volver). Se mantuvo lo confirmado, pero el documento deja el plan para la etapa 7: probar, y si la dirección se siente lenta, pasar a 60 Hz y 50 ms cambiando dos constantes; la predicción queda como último recurso.
3. **El servidor no puede cargar el circuito.** Al revisar el código actual contra el diseño, se vio que cargar el mapa de Tiled crea texturas, algo que en LibGDX solo se puede hacer en el hilo de render, no en un hilo del servidor. Se resolvió en el diseño: el host carga el `Circuito` y se lo pasa al servidor al crearlo.
4. **Decimales y configuración regional.** En una PC configurada en español, `String.format` escribe los decimales con coma, y el otro lado no los puede leer como número. Se decidió que todos los tiempos de los mensajes TCP vayan en milisegundos enteros.
5. **Diagramas.** Los tres diagramas Mermaid del documento (arquitectura, estados y secuencia) se renderizaron con la biblioteca oficial de Mermaid para confirmar que no tienen errores de sintaxis antes de subirlos. GitHub los muestra como gráficos.

Cambios al modelo que quedan para la etapa 7

El documento termina con una lista de 7 cosas del código actual que complican la red. Entre ellas, que la pantalla de carrera simula y dibuja a la vez (en red el cliente solo dibuja), que `VistaAuto` y la cámara necesitan un `Auto` con física, que hace falta marcar un jugador como desconectado y que el color del auto sale del id del jugador y no del auto elegido en el lobby.

Próximo paso

La guía indica que los cinco integrantes lean docs/PROTOCOLO.md antes de seguir con la etapa 6 (conexión y lobby por TCP), porque es lo que tienen que poder explicar.

Commits

- Documentación de la etapa en la rama `docs/protocolo-red`: el protocolo, esta página y la actualización del PDF.

Etapa 6: Conexión y lobby por TCP

Objetivo

Implementar la primera parte del protocolo de red (docs/PROTOCOLO.md, secciones 1, 2, 3, 7 y 8): que una PC cree una partida, que otras se unan por IP, y que todos vean en vivo el lobby, elijan auto, marquen "listo" y el host inicie. Todavía sin UDP y sin carrera sincronizada: al iniciar, cada uno corre su propia carrera local.

El plan (se mostró y se confirmó antes de programar)

El grupo confirmó tres decisiones que el plan proponía:

1. **"Crear partida" pide el nombre.** La guía no lo decía, pero el host necesita uno para aparecer en el lobby.
2. `INICIAR` **exige al menos 2 jugadores,** como dice el protocolo (`Config.MIN_JUGADORES`). Para probar solo hace falta abrir 2 instancias.
3. **Al iniciar, cada cliente corre su propia carrera local** con su nombre y su color, hasta que en la etapa siguiente se sincronice.

Qué se hizo

Paquete `red` **(Java puro, sin LibGDX):**

Clase	Qué hace
`Protocolo`	Nombres de mensajes y de errores como constantes; arma y lee las líneas (`;`, UTF-8); valida el nombre (solo letras, números y espacios, de 1 a 12 caracteres)
`ConexionTcp`	Un `Socket` con mensajes por línea: escritura sincronizada con `flush`, `setTcpNoDelay(true)`, timeout de lectura de 6 s
`servidor/ServidorPartida`	`ServerSocket` en el puerto 7777 con tres tipos de hilos: uno que acepta, uno lector por cliente y uno de lógica
`cliente/ClientePartida`	Se conecta en segundo plano (timeout de 3 s), lee en otro hilo y deja los mensajes en una cola thread-safe; manda un `PING` cada 2 s
`EstadoLobby`	La foto del lobby (quién es el host y, por jugador, id, nombre, auto y si está listo), que sabe armarse y leerse como mensaje `LOBBY`
`SesionRed`	Lo que una PC tiene abierto: su cliente y, si es el host, también el servidor
`DireccionesLocales`	Las IPv4 de la red local de esta PC (sin las de máquinas virtuales), para mostrarlas en el lobby del host
`RegistroRed`	Log en consola de cada mensaje enviado y recibido, controlado por `Config.DEBUG_RED`

Lo que implementa el servidor: `UNIRSE` con las cinco validaciones de error (versión, carrera empezada, partida llena, nombre inválido y nombre repetido), `ELEGIR_AUTO` sin repetir, `LISTO`, `INICIAR` (solo el host y con todos listos), `SALIR`, `PING` / `PONG`, las caídas de un cliente (se libera su lugar y se avisa) y el cierre del host (`CERRADA` para todos). Al iniciar manda `CUENTA_REGRESIVA` y, a los 3 s, `LARGADA`; desde ese momento rechaza a quien quiera entrar.

Pantallas:

- **Crear partida:** pide el nombre, levanta el servidor y se conecta a `127.0.0.1`. Si el puerto 7777 está ocupado, muestra el error.
- **Unirse:** nombre e IP. Muestra "Conectando..." y, si falla, un mensaje claro (no se pudo conectar, partida llena, versión distinta, nombre repetido...).
- **Lobby en vivo:** los 5 lugares con su color, nombre y estado; cinco cuadros de color para elegir auto (los que tomó otro jugador quedan deshabilitados y muestran su inicial); botón "Estoy listo"; y, solo para el host, "Iniciar carrera" (deshabilitado hasta que haya 2 jugadores y todos estén listos) más sus IPs de la red para dictárselas al resto.
- **Al iniciar:** todos pasan a la carrera con el nombre y el color elegidos. Si se corta la conexión o el host cierra la partida, vuelven al menú con un aviso.

Decisiones

- **Un solo hilo toca el estado de la partida** (el de lógica del servidor). Los hilos lectores le dejan eventos en una `LinkedBlockingQueue` y él los procesa en orden, así no hacen falta `synchronized` sobre los jugadores. La única excepción es el `PING`, que el lector contesta en el momento porque no toca ningún estado.
- **En el cliente, la red nunca toca LibGDX.** El hilo de red solo encola mensajes; cada pantalla los procesa al principio de su `render()`. Para eso se agregó a `PantallaBase` un gancho `actualizar(delta)`.
- **El host es el primer jugador que se une.** Es el que crea la partida y se conecta a su propio servidor de inmediato. Se aclaró en el protocolo.
- **No se activa `SO_REUSEADDR` en el servidor.** En Windows permitiría que otro programa abra el mismo puerto; sin eso, una segunda partida en la misma PC falla con un error claro en lugar de quedar rota.
- **Los mensajes de la red se traducen a un `Mensaje`.** El cliente también genera dos mensajes propios (`_CONEXION_FALLIDA` y `_CONEXION_PERDIDA`, con un guion bajo para no confundirse con los del protocolo), así las pantallas tratan igual lo que llega de la red y las fallas locales.
- **La conexión se mantiene viva también en la carrera y en los resultados.** Si esas pantallas no mandaran `PING`, el servidor daría de baja al jugador a los 6 s. Se verificó (ver abajo).

Problemas encontrados y cómo se resolvieron

1. **La primera corrida de la prueba dio un fallo, que era un error de la prueba y no del servidor.** El servidor había aceptado el cambio de auto y difundido el lobby correcto (se comprobó en el log). La prueba leyó por error un mensaje `LOBBY` viejo que todavía estaba en la cola del cliente, que es una cola en orden de llegada. Se corrigió la prueba para que espere a que se vea el cambio.

2. **"Conectado a la partida de ?".** Al mirar la captura del lobby de un invitado, el texto con el nombre del host se calculaba antes de que llegara el primer `LOBBY`. Se cambió para que se actualice cada vez que cambia el lobby.
3. **Un mensaje de error largo se salía del borde** de la pantalla en el formulario de unirse; se acortó.
4. **Una decisión que evitó un error futuro:** al iniciar, cada cliente corre una carrera local que dura unos 35 segundos y la sesión seguía abierta; sin `PING` en esas pantallas, el servidor habría dado de baja a todos a los 6 segundos. Por eso `PantallaCarrera` y `Resultados` también atienden la red.

Verificación

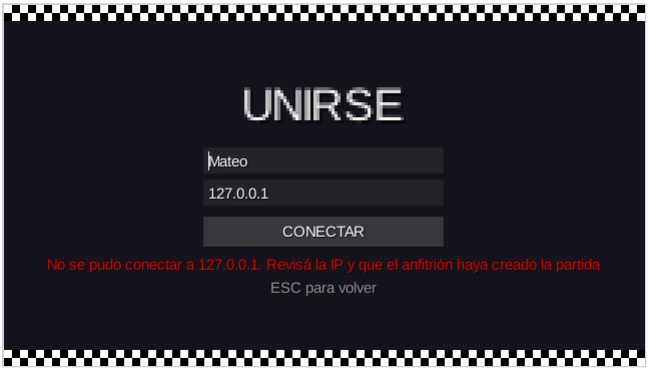
Capa de red, con un servidor real y clientes falsos (una prueba descartable, sin pantalla). 40 comprobaciones, todas correctas:

Grupo	Qué se comprobó
Lobby	El host recibe el id 0 y se lo marca como host; un nombre con tilde (`Joaquín`) llega bien por UTF-8; el segundo jugador recibe el id 1 y un auto distinto
Errores de `UNIRSE`	Nombre repetido (sin distinguir mayúsculas), nombre con un símbolo, nombre de 13 caracteres, versión distinta, partida llena (el sexto jugador) y carrera ya empezada, todos con su código de error; el servidor cierra la conexión después de rechazar
Autos	Cada jugador recibe un auto distinto; elegir el auto de otro da `AUTO_OCUPADO` ; un auto que se liberó se puede elegir
Inicio	`INICIAR` de alguien que no es host da `NO_PERMITIDO` ; con jugadores sin marcar da `FALTAN_JUGADORES` ; con un solo jugador tampoco deja iniciar; con todos listos, todos reciben `CUENTA_REGRESIVA` , y la `LARGADA` llega a los **3.009 ms**
Durante la carrera	Un jugador que sale genera `SALIO` para los demás; elegir auto da `NO_PERMITIDO`
Cierre	Cuando el host sale, todos reciben `CERRADA` y después ven la conexión cerrada; si el host cierra con `SALIR` , pasa lo mismo
Caídas	Un cliente que se corta sin avisar libera su lugar en el lobby; un cliente **mudo** se da de baja a los **6.044 ms** (el timeout es 6.000); los que mandan `PING` siguen conectados después de 9 s
Robustez	Líneas vacías, mensajes desconocidos, campos vacíos, un `LOBBY` mandado por un cliente y un `UNIRSE` sin campos: el servidor no se cae y sigue contestando el `PING`
Fallas de conexión	Un puerto cerrado da `CONEXION_FALLIDA` en 12 ms; una IP que no responde, en 2.862 ms (el timeout es 3.000); un puerto ocupado lanza `BindException`
Formato	El ejemplo de mensaje `LOBBY` del protocolo se lee bien y armar y leer es reversible

Interfaz de punta a punta, con la ventana oculta, usando el flujo real y tres jugadores falsos que se conectan al servidor del host: el menú muestra el aviso; unirse a un puerto vacío muestra el error; un nombre inválido muestra su mensaje; "Crear" lleva al lobby con el host; entran 3 jugadores; el host marca listo y el botón de iniciar se habilita; al iniciar pasa a la carrera; **a los 7,8 s de carrera sigue conectado** (el `PING`

funciona); un jugador sale y el host sigue en la carrera; al volver al menú con ESC, los demás reciben `CERRADA`; y un invitado ve el lobby y sale con ESC.

Capturas

Formulario de unirse con un error	Lobby del anfitrión
	
Todos listos: se habilita "Iniciar"	**Lobby de un invitado**
	

Cómo probarlo

Tres instancias en la misma PC (Eclipse, `Lwjgl3Launcher`, Run As, tres veces):

1. En la primera: "Crear partida", con un nombre.
2. En la segunda y la tercera: "Unirse a una partida", con otro nombre cada una y la IP `127.0.0.1`.
3. En las tres: elegir un auto y "Estoy listo". En la primera, "Iniciar carrera".
4. Con la consola de Eclipse se ve el log de cada mensaje (`Config.DEBUG_RED`).

Dos PC: la PC del host muestra en el lobby su IP de la red (por ejemplo `192.168.1.46`); la otra la escribe en "Unirse". Si no conecta, revisar el firewall de Windows de la PC del host (tiene que permitir Java en redes privadas, o el puerto TCP 7777) y que la red del colegio no aísle a los equipos entre sí. Desde la otra PC se puede comprobar con `Test-NetConnection 192.168.1.46 -Port 7777` en PowerShell. La guía recomienda probar cuanto antes en las PC del colegio: si la red bloquea algo, mejor enterarse ahora.

Commits

- Los commits de la etapa están en la rama `feature/lobby-tcp`.

Etapa 7: Carrera sincronizada por UDP

Objetivo

Que la carrera se juegue en red de verdad, según docs/PROTOCOLO.md (secciones 4 a 8). La simula solo el servidor, con la última entrada recibida de cada jugador, y manda el estado de todos los autos por UDP. Los clientes mandan lo que aprieta cada jugador y dibujan lo que llega, interpolado. Al terminar vienen los resultados por TCP y la opción de revancha, y la prueba local tiene que seguir andando sin red. La guía la marca como "la parte que define si se aprueba la materia". Se usó Claude Opus 5.5, como recomienda la guía.

El plan (se mostró y se confirmó antes de programar)

La idea central fue que **una sola pantalla de carrera dibuje "fotos" de la carrera sin importar de dónde vengan**: de una simulación local o del servidor. El grupo confirmó tres puntos:

1. **Resultados y revancha:** solo el host puede pedir la revancha, y si el host vuelve al menú se cierra la partida para todos.
2. **Una desviación del protocolo:** el color de cada auto lo resuelve la pantalla con el lobby (id del jugador → auto elegido), en lugar de guardarlo en el modelo. El protocolo se actualizó con esta decisión.
3. **Un extra para la demo:** la tecla F8 cambia en vivo la pérdida de paquetes simulada (0 %, 20 %, 50 %), para mostrar sin recompilar que el juego la tolera.

Qué se hizo

Clases nuevas:

Clase	Qué hace
`juego/FotoCarrera`, `juego/FotoAuto`	La carrera en un instante: estado, reloj (negativo en la cuenta regresiva), cierre y, por auto, posición, ángulo, vueltas, posición en carrera y tiempos. Es lo mismo que viaja en el `ESTADO`
`juego/FuenteCarrera`	Lo que consume la pantalla: avanzar con las teclas, devolver la foto, los resultados y el nombre y color de cada id
`juego/SimulacionLocal`	La prueba local de antes (la carrera simulada en la PC, a paso fijo)
`red/PaquetesUdp`	Arma y lee `ENTRADA` (11 bytes) y `ESTADO` (132 bytes con 5 autos) con `ByteBuffer`, byte por byte como en el protocolo
`red/PerdidaSimulada`	Descarta al azar la fracción de paquetes UDP que indique `Config.PERDIDA_UDP_SIMULADA` (o F8)
`red/cliente/Interpolador`	Búfer de estados, descarte de atrasados y duplicados, reloj estimado del servidor y foto interpolada 100 ms en el pasado

Clase	Qué hace
`red/cliente/CarreraEnRed`	La carrera vista desde un cliente: manda `ENTRADA` 60 veces por segundo y pasa los `ESTADO` al interpolador

Cambios en lo que ya existía:

- `ServidorPartida`**:**
  - Suma un socket UDP en el puerto 7778 y un hilo receptor que valida cada `ENTRADA` (tamaño, tipo, id, token y secuencia), guarda la última de cada jugador y aprende su dirección.
  - Su hilo de lógica pasó a ser un ciclo de **60 ticks por segundo**, que simula la `Carrera` y manda el `ESTADO` a todos uno de cada dos ticks. También avisa por TCP la `LARGADA` y los `RESULTADOS`, y atiende la `REVANCHA`.
  - La cuenta regresiva ahora la lleva la `Carrera` y no un temporizador aparte.
  - Recibe el `Circuito` ya cargado, porque el mapa de Tiled solo se puede cargar en el hilo de render.
- `ClientePartida`**:** abre su socket UDP en un puerto libre al recibir la `BIENVENIDA`, con un hilo que acepta solo los `ESTADO` que vienen de la dirección del servidor.
- `Carrera`**:** suma la foto, el reloj con decimales y `marcarDesconectado`. El auto del que se fue no se simula, no choca, no se dibuja, no cuenta para "todos terminaron" y queda último si no había terminado.
- `PantallaCarrera`**:** lee el teclado, se lo pasa a la fuente y dibuja la foto. Hay dos formas de crearla: local (2 jugadores con pantalla dividida) y en red (1 jugador y su cámara). En red también atiende lo que llega por TCP: resultados, abandonos, revancha y cierre.
- `VistaAuto` **y** `CamaraSeguimiento`**:** reciben datos simples (posición, ángulo, velocidad) en lugar de un `Auto` con física, que el cliente en red no tiene.
- `Resultados`**:** en red, el host ve "Revancha" y "Cerrar la partida" y los demás esperan su decisión.
- `Lobby`**:** con `CUENTA_REGRESIVA` abre la carrera en red; desaparece la carrera local temporal de la etapa 6.

Recorrido de una tecla

Qué pasa desde que un jugador aprieta el acelerador hasta que su auto se mueve en la pantalla de otro:

1. **Teclado (cliente A, hilo de render).** `PantallaCarrera` lee las teclas en cada cuadro y arma una `EntradaAuto` (acelerar, frenar, giro).
2. **Envío (cliente A).** `CarreraEnRed` la manda 60 veces por segundo: `PaquetesUdp` la convierte en un `ENTRADA` de 11 bytes, con el id, el token de la bienvenida, una secuencia que sube de a uno y los botones como bits. Sale por UDP al puerto 7778 del host.
3. **Recepción (servidor, hilo receptor UDP).** El servidor valida el paquete y descarta el que tenga el token equivocado o una secuencia menor o igual a la última. Si es bueno, guarda los botones como "última entrada" de ese jugador (un campo `volatile`) y anota de qué dirección vino.
4. **Simulación (servidor, hilo de simulación).** En el próximo tick (cada 16,7 ms) arma el mapa de entradas con la última de cada jugador y llama a `carrera.actualizar(entradas, 1/60)`. El auto de A acelera: esa es la única carrera que existe de verdad.
5. **Estado (servidor).** Uno de cada dos ticks, `PaquetesUdp` arma un `ESTADO` con todos los autos (132 bytes con 5) y lo manda a la dirección que cada jugador usó para escribirle.

6. **Recepción (cliente B, hilo receptor UDP).** Acepta el paquete solo si viene del servidor, lo lee y lo deja en una cola thread-safe.
7. **Interpolación (cliente B, hilo de render).** Al principio del cuadro, `CarreraEnRed` vacía la cola en el `Interpolador`, que descarta los estados viejos o repetidos por su tick. La pantalla le pide la foto del momento "ahora menos 100 ms" y el interpolador calcula un punto entre los dos estados que rodean ese momento.
8. **Dibujo (cliente B).** `PantallaCarrera` dibuja cada auto de la foto con `VistaAuto`: el de A aparece moviéndose.

Medido de punta a punta, del paso 1 al 8 pasan **unos 97 ms** (ver "Verificación").

Decisiones

- **Una sola pantalla para la prueba local y para la red.** Separar "de dónde sale la foto" de "cómo se dibuja" evitó tener dos pantallas casi iguales, y es lo que garantiza que la prueba local siga andando.
- **El servidor programa los ticks sobre un horario fijo.** Si el sistema operativo despierta al hilo un poco tarde, el siguiente tick se adelanta, y el promedio queda en 60 por segundo. Mientras espera, procesa los mensajes TCP que lleguen.
- **La entrada se manda como estado, no como evento.** Perder una `ENTRADA` no importa: la siguiente, 17 ms después, trae los mismos botones.
- **El cliente solo acepta `ESTADO` de la dirección del servidor,** y el servidor solo acepta `ENTRADA` con el token correcto. Un paquete de otra partida o de una conexión anterior no se confunde con uno válido.
- **Se mantuvo 30 estados por segundo con 100 ms de retraso de dibujo,** porque la demora medida (97 ms) resultó menor que la estimada (135 ms).

Problemas encontrados y cómo se resolvieron

1. **Cuatro comprobaciones del interpolador fallaron, pero por un error de la prueba.** La prueba agregaba estados seguidos sin dejar pasar el tiempo entre uno y otro. El interpolador corrige su reloj de a poco (un 10 % por estado) para no dar saltos con la variación normal de la red, así que se quedaba atrás. En el juego real el tiempo pasa entre estado y estado; con la prueba corregida, las 4 dieron bien.
2. **La primera medición de la demora (159 ms) estaba inflada por la física.** Se esperaba que el auto se corriera 0,5 px, y un auto que arranca de parado tarda unos 67 ms de pura física en recorrer eso. Se separó la medición: la demora de la red se mide con el primer cambio de posición (unos 97 ms), y aparte se mide cuándo el auto se corre 1 px en pantalla (unos 185 ms, contra 100 ms de la física sola sin red).
3. **El aviso "X abandonó la carrera" tapaba el auto propio,** porque salía en el centro de la pantalla, que es donde la cámara pone al auto. Se movió arriba, debajo del HUD.
4. **Un detalle de la prueba, no del juego:** en la prueba con interfaz, el auto del host queda quieto porque nadie aprieta las teclas de esa ventana. Por eso la carrera terminó por el cierre de 30 s. Sirvió para comprobar que el cierre y "no terminó" funcionan en red.

Verificación

Paquetes e interpolador (sin red):

- `ENTRADA` mide 11 bytes y `ESTADO` con 5 autos mide 132, y los dos se leen igual a como se armaron.
- Un paquete dañado o de otro tipo se rechaza.
- El interpolador:
  - calcula el punto medio entre dos estados;
  - estima la velocidad;
  - gira por el camino corto (de 350° a 10° pasa por 0°);
  - descarta duplicados y atrasados;
  - se queda en el último estado si dejan de llegar, sin extrapolar;
  - corrige de golpe un salto grande de reloj.

Carrera real por UDP, con un servidor y tres clientes en la misma PC. Cada cliente maneja con un piloto automático que mira su propia foto interpolada, igual que un jugador mira su pantalla. Las 43 comprobaciones dieron bien:

Qué	Resultado
Cuenta regresiva	Llegan estados con los autos en la grilla; el host está en el lugar 1; acelerar no mueve el auto hasta la largada
Demora de la red (6 mediciones)	90 a 104 ms, **unos 97 ms en promedio**, en la pantalla de otro jugador y en la propia
Frecuencia	**30,0 estados por segundo**, sin ninguno descartado por secuencia (en la misma PC no se desordenan)
Token falso	60 `ENTRADA` con el token equivocado: el auto no se mueve
Secuencia vieja	60 `ENTRADA` con el token correcto pero secuencia vieja: se descartan
`ESTADO` falso	20 paquetes desde otra dirección con los autos en otra posición: el cliente los descarta todos
Abandono en carrera	Los demás reciben `SALIO`, el auto figura como desconectado y queda último sin tiempo total
Fin	Los dos que siguen terminan (47,5 y 47,8 s) y los dos clientes reciben exactamente los mismos resultados
Revancha	Solo la puede pedir el host; vuelven al lobby 2 jugadores, sin el que abandonó y sin nadie listo
Con 20 % de pérdida simulada	La segunda carrera se completa (33,4 y 34,0 s); llegan 23,9 estados por segundo en vez de 30, y el juego sigue andando
Cierre	Cuando el host se va, el otro jugador se entera

Interfaz de punta a punta, con la ventana oculta, en tres escenarios:

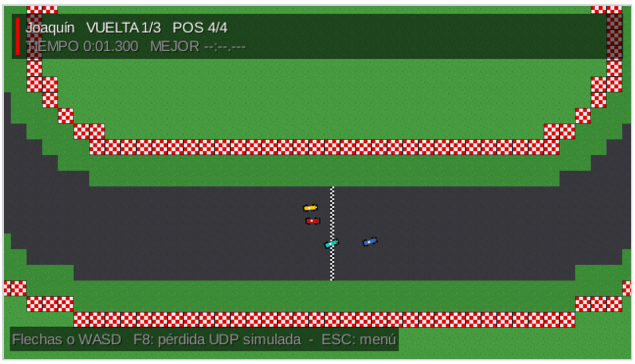
1. **La pantalla del host con 3 jugadores de prueba:** cuenta regresiva, largada, el indicador de pérdida al 20 %, el aviso de abandono, los resultados, la revancha y la vuelta al lobby. Al cerrar la partida, los demás se enteran.

2. **La pantalla de un invitado** que se une a la partida de otro: recibió 210 estados en los primeros segundos y, cuando el anfitrión se fue, volvió al menú con el aviso.
3. **La prueba local** con 2 jugadores sigue andando.

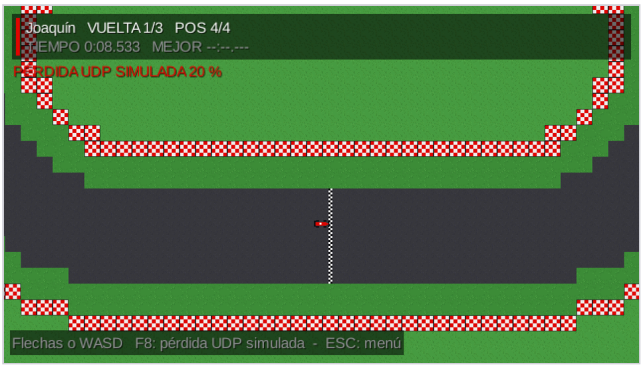
Además se volvió a correr la prueba de la etapa 6 (lobby por TCP) contra el servidor nuevo: 40 de 40.

Capturas

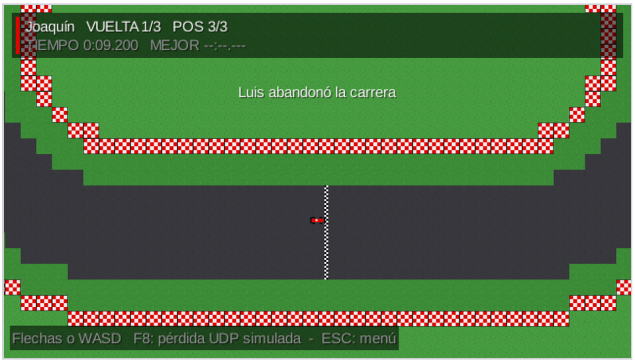
Largada en red (vista del host)



Pérdida simulada al 20 %



Aviso de abandono



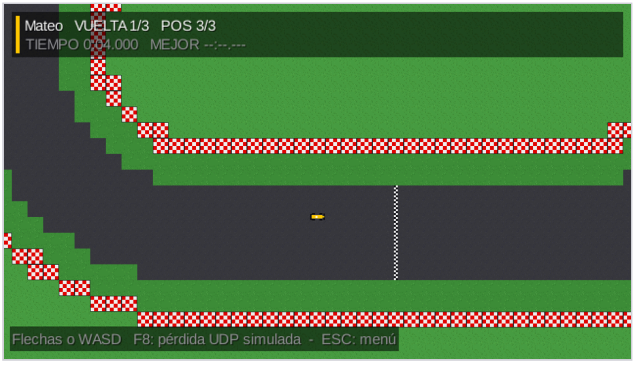
Resultados (host)



Lobby después de la revancha



Carrera vista por un invitado



Cómo probarlo

En una PC, con Eclipse:

1. Corré `Lwjgl3Launcher` 2 o 3 veces. En una: "Crear partida"; en las otras: "Unirse" con la IP `127.0.0.1`.
2. Elegí auto, "Estoy listo" en todas e "Iniciar carrera" en la del host.
3. Cada ventana maneja su auto (flechas o WASD) y ve a los demás moverse.
4. Para ver la tolerancia a pérdida: F8 en plena carrera (20 % y 50 %).

En dos PC: como en la etapa 6. Además del TCP 7777, el firewall de la PC del host tiene que dejar pasar el UDP 7778. Si en la PC del invitado el cartel queda en "Esperando al servidor..." después de la largada, el UDP está bloqueado. Los avisos claros para ese caso son parte de la etapa 8.

Lo que queda para la etapa 8

La guía reserva para la etapa 8 la robustez: un aviso y la vuelta al menú si dejan de llegar estados, y que cerrar la ventana en cualquier pantalla libere los sockets y los hilos.

Commits

- Los commits de la etapa están en la rama `feature/carrera-udp`.

Etapa 8: Robustez de red

Objetivo

Que ninguna falla de red deje al juego colgado o muestre un error sin explicar: cada caso tiene que terminar con un mensaje claro en pantalla, sin excepciones sin atrapar ni hilos sueltos. La guía pide resolver la desconexión de un cliente en el lobby o en la carrera, el cierre del host, que no lleguen estados UDP durante un tiempo (`Config.TIMEOUT_RED`), el puerto ocupado, la IP inválida o inalcanzable, la versión distinta, el nombre vacío o repetido, y que cerrar la ventana en cualquier pantalla libere sockets e hilos. Y explicar cómo reproducir cada caso a mano.

El plan (se mostró y se confirmó antes de programar)

Primero se revisó qué ya estaba resuelto por las etapas 6 y 7 (la caída de un cliente, el cierre del host, el puerto ocupado, la versión, los nombres) y se decidió volver a verificarlo en lugar de darlo por sentado. Lo que faltaba se resolvió en cuatro puntos. El grupo confirmó dos decisiones: `TIMEOUT_RED` de 5 segundos, con un aviso de "conexión inestable" a 1 segundo; y que una falla inesperada del servidor cierre la partida para todos (es lo más seguro: una partida con la simulación muerta no se puede continuar).

Qué se hizo

1. **Sin estados UDP.** La pantalla de carrera en red cuenta cuánto pasa sin recibir estados. A 1 segundo muestra "Conexión inestable...", y a `Config.TIMEOUT_RED` (5 s) vuelve al menú con un aviso. Si nunca llegó ninguno, el mensaje dice lo más probable: "*No llegan datos por UDP: revisá el firewall de la PC del host (puerto 7778)*". Si se cortó a mitad de carrera: "*Se cortó la conexión con la carrera: no llegan datos hace 5 s*". No se controla una vez terminada la carrera, porque el servidor deja de mandar estados. Si la ventana se congela un rato, el reloj no acumula más de 0,25 s por cuadro, para que los estados atrasados que llegan enseguida no se tomen por una falla.
2. **Aviso en el servidor.** Si a los 5 segundos de empezar un jugador no mandó ninguna entrada UDP, la consola del host lo dice, con el nombre del jugador y el puerto: es el mismo problema de firewall visto desde el otro lado.
3. **Ningún hilo muere en silencio.**
   - Si falla la simulación del servidor, se registra el error y se cierra la partida con el motivo (por ejemplo, "*Error interno del servidor (IllegalStateException)*") para todos.
   - Un mensaje TCP que rompa el procesamiento da de baja solo a ese cliente.
   - Un paquete UDP defectuoso se ignora sin detener la recepción, tanto en el servidor como en el cliente.
4. **Cerrar la ventana a prueba de fallas.** `Main.dispose` hace cada paso por separado (ocultar la pantalla, cerrar la partida en red, liberar la pantalla y los recursos), y `SesionRed.cerrar` cierra el servidor aunque falle el cliente. Además, `ClientePartida.cerrar` corta un intento de conexión en curso en vez de esperar el timeout.

5. **Tope de espera al conectar.** El formulario de unirse deja de esperar a los 5 segundos aunque la resolución de un nombre tarde más que el timeout de conexión.
6. **Extra para la demo:** la tecla F8 llega también a 100 % de pérdida simulada (corte total), así se puede mostrar en vivo el aviso y la vuelta al menú sin tocar el firewall.

Decisiones

- **Volver al menú y no reintentar.** Un corte de 5 segundos en un juego de carreras ya arruinó la partida; reconectar a mitad de carrera es mucho más complejo y no está en el alcance.
- **Un mensaje distinto para cada causa probable.** Ver que nunca llegó ningún estado (casi seguro el firewall) es distinto de que se cortó una conexión que andaba, y al jugador se le dice lo que puede hacer.
- **Las fallas del servidor cierran la partida** en lugar de intentar seguir con un estado dudoso.

Problemas encontrados y cómo se resolvieron

1. **Un hilo tardaba en morir al cerrar la ventana mientras se conectaba.** La prueba de cierre en cada pantalla mostró que, con un intento de conexión en curso, el hilo de conexión seguía vivo hasta que vencía el timeout de 3 s. No colgaba el programa (es un hilo secundario), pero se corrigió: `cerrar` ahora cierra el socket que se está conectando y el hilo termina al instante.
2. **La primera corrida del cierre en la pantalla del invitado se colgó, pero era un error de la prueba.** Dos jugadores de prueba se conectaron en paralelo y el que quedó como host no era el que mandaba `INICIAR`, así que el servidor rechazó el pedido, como corresponde. Se corrigió la prueba para que inicie el jugador que sea host.
3. **La primera medición de las IP mal escritas era inválida.** El reloj de la prueba arrancaba al armar el paso y no al ejecutarlo. Al corregirla se vio el comportamiento real: todas fallan en menos de 3 segundos.

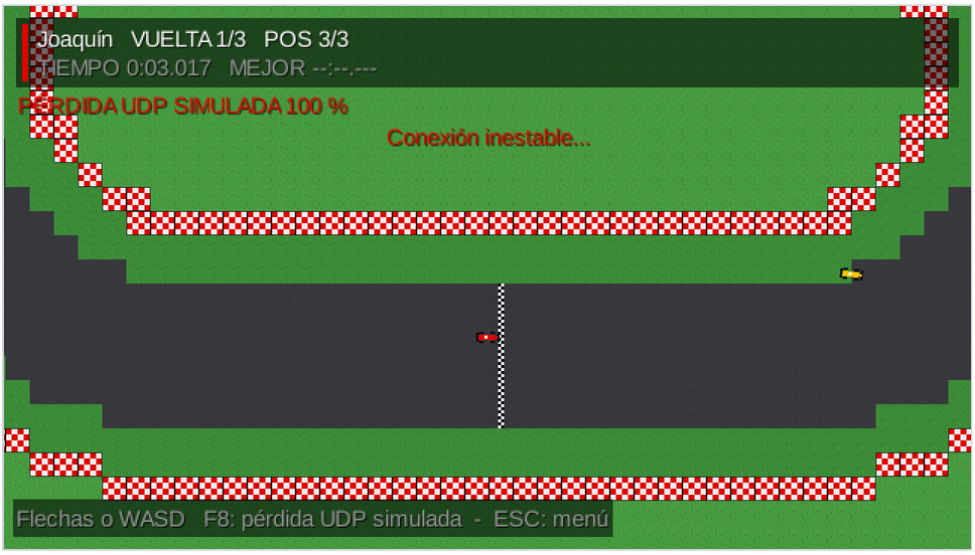
Verificación

Caso	Cómo se comprobó	Resultado
UDP bloqueado desde el principio	Pérdida simulada al 100 % antes de iniciar, con la pantalla real y jugadores de prueba	A 1,5 s, "Conexión inestable..."; vuelve al menú a los **4.996 ms** con el mensaje del firewall; la sesión queda cerrada. El servidor avisa en su consola por cada jugador
UDP que se corta a mitad de carrera	Corte total después de 3 s de carrera	Aviso a 1,4 s del corte; vuelve al menú a los **5.034 ms**
Pérdida parcial y cortes cortos (falsas alarmas)	8 s con 50 % de pérdida; después 3 s de corte total y vuelta de los datos	Se queda en la carrera; con el corte muestra el aviso y al volver los datos, se borra solo
Falla inesperada del servidor	Se provoca una excepción dentro de la simulación	Todos reciben `CERRADA` con el motivo en **23 ms** y el servidor queda inactivo
IP mal escrita	`abc`, `999.1.1.1`, `192.168.1`, `a b`, una IP que no responde	Todas fallan con mensaje claro en **menos de 3 segundos** (`abc` en 2,3 s por la búsqueda de nombre)

Caso	Cómo se comprobó	Resultado
Cerrar la ventana en 7 pantallas	Se abre el juego real con la ventana oculta en cada pantalla y se cierra; se revisan hilos vivos y puertos	En **las 7** el proceso termina solo, no quedan hilos de red y los puertos TCP 7777 y UDP 7778 quedan libres. Las pantallas: menú, conectando, lobby del host, carrera del host, resultados, lobby de invitado y carrera de invitado
Lo ya hecho en las etapas 6 y 7	Se repiten sus pruebas	Lobby por TCP: 40 de 40. Carrera por UDP: 43 de 43

Capturas

Vuelve al menú por el firewall	Vuelve al menú por un corte
	



Aviso de conexión inestable durante la carrera

Cómo reproducir cada caso a mano

Se necesita el proyecto abierto en Eclipse y `Lwjgl3Launcher` ejecutado 2 o 3 veces (una ventana por jugador), o dos PC.

Caso	Cómo reproducirlo	Qué se tiene que ver
Un cliente se cae en el lobby	Con dos ventanas en el lobby, cerrar una con la X (o pararla desde Eclipse con el botón rojo)	En la otra, el jugador desaparece de la lista al instante. Con cable desconectado, a los 6 s
Un cliente se cae en la carrera	Igual, pero con la carrera en marcha	En las otras, el aviso "X abandonó la carrera" y su auto desaparece; en los resultados figura sin tiempo total
El host cierra el juego	Con la carrera o el lobby en marcha, cerrar la ventana del host	Los demás vuelven al menú con "La partida se cerró: ..."
No llegan estados UDP a mitad de carrera	En una ventana en carrera, apretar **F8** hasta que diga 100 %	A 1 s, "Conexión inestable..."; a los 5 s vuelve al menú con el aviso. Con F8 de nuevo antes de 5 s (o apretándolo hasta 0 %), sigue la carrera
El firewall bloquea el UDP	En la PC del host, crear una regla de entrada que bloquee el UDP 7778 y probar con otra PC. O, más simple, poner 100 % de pérdida antes de iniciar	El invitado, "Esperando al servidor...", "Conexión inestable..." y a los 5 s el menú con "No llegan datos por UDP: revisá el firewall...". En la consola del host, la línea `AVISO: no llegó ninguna ENTRADA UDP de ...`
Puerto ocupado al crear la partida	Con una partida ya creada en la PC, crear otra desde una segunda ventana	Mensaje "No se pudo crear la partida: los puertos 7777 y 7778 están ocupados..." y el formulario sigue abierto
IP inválida o inalcanzable	En "Unirse", escribir `abc`, `999.1.1.1` o una IP de la red que no exista	En menos de 3 segundos, "No se pudo conectar a ... Revisá la IP y que el anfitrión haya creado la partida"
IP vacía	En "Unirse", dejar la IP en blanco	"Escribí la IP del anfitrión"
Versión de protocolo distinta	Cambiar `Config.VERSION_PROTOCOLO` a 2 en una de las PC (o ventanas) y volver a compilar	Al unirse, "Tu juego es de otra versión (2, la de la partida es 1)"
Nombre vacío, inválido o repetido	Dejar el nombre vacío; escribir uno con símbolos o de más de 12 letras; o unirse con el mismo nombre que otro jugador (aunque cambien las mayúsculas)	"El nombre debe tener de 1 a 12 letras, números o espacios", o "Ya hay un jugador con ese nombre"

Caso	Cómo reproducirlo	Qué se tiene que ver	
Partida llena / carrera empezada	Con 5 jugadores en el lobby, intentar unirse con un sexto; o unirse cuando ya empezó la carrera	"La partida está llena" / "La carrera ya empezó"	
Cerrar la ventana en cualquier pantalla	Cerrar con la X desde cada pantalla (menú, conectando, lobby, carrera, resultados)	El programa termina solo. Se puede comprobar que el puerto quedó libre con `netstat -ano \`	findstr 7777 en la consola de Windows (no debe aparecer nada) y que ya no hay un proceso java` de más en el Administrador de tareas
Falla interna del servidor	No se puede provocar a mano con el juego normal; se comprobó con una prueba automática que inyecta la excepción	Todos vuelven al menú con "La partida se cerró: Error interno del servidor (...)"	

Commits

- Los commits de la etapa están en la rama `feature/robustez-red`.

Etapa 9: Pruebas en LAN real

Objetivo

Comprobar que el juego funciona entre computadoras distintas y no solo con varias ventanas en una misma PC. La guía pide generar el ejecutable, copiarlo a cada PC, probar primero con equipos propios (router u hotspot de un celular) y después en las PC del colegio, con 2, 3 y 5 jugadores, y también con pérdida de paquetes UDP simulada (`Config.PERDIDA_UDP_SIMULADA` en 0,1).

Esta etapa no tuvo código nuevo: es una prueba manual hecha el 29/09/2026 en el colegio. Por eso se documenta con lo que se vio y lo que quedó sin probar.

Qué se hizo

1. Se llevó el juego en un pendrive, como ejecutable (`.exe`) generado con el proyecto, y se copió a las notebooks.
2. Se intentó jugar en la red del colegio, con las computadoras conectadas a la LAN.
3. Como no se pudo, se repitió la prueba con dos notebooks del colegio conectadas al punto de acceso (hotspot) de un celular.
4. En esa red se jugó una carrera completa y se probó la pérdida de paquetes simulada.

Resultados

Prueba	Resultado
PC del colegio conectadas a la LAN	**No funcionó.** La partida se podía crear, pero al unirse desde otra PC el juego respondía que no se podía encontrar la IP y no se llegaba al lobby
2 notebooks en el hotspot de un celular	**Funcionó.** Una hizo de host y la otra se unió. Se jugó una carrera completa, con resultados y revancha, sin lag ni tirones que se notaran
Windows Defender Firewall en las notebooks del hotspot	No apareció ningún cartel pidiendo permiso para Java en la red
Pérdida UDP simulada	Con pérdidas parciales el juego siguió normal. Con 100 % (corte total), el jugador afectado salió de la partida por falta de datos, como está diseñado en la etapa 8, y después volvió a jugar con normalidad

Qué pasó en el colegio

Se pudo crear la partida, pero nadie pudo unirse. Eso es lo que se espera cuando algo bloquea las conexiones entrantes hacia la PC del host: el host se conecta a sí mismo por `127.0.0.1`, que no sale a la red, así que crear la partida anda igual. Del lado del invitado, el intento de conexión TCP al puerto 7777 no llega y falla con el mensaje de "No se pudo conectar...".

Las computadoras tenían firewall y el grupo no tenía permisos de administrador en ellas, así que no se pudo desactivar ni crear una regla para los puertos 7777 (TCP) y 7778 (UDP). Con los datos que hay no se puede distinguir si lo que bloquea es el firewall de cada equipo o un aislamiento de la red del colegio entre equipos (la guía menciona las dos posibilidades). No se probó cuál de los dos era.

Cómo se resolvió

Se usó el hotspot de un celular, que es el plan B que preveía la guía. Ahí no hizo falta tocar ningún firewall (Windows ni siquiera mostró el cartel de permiso) y la partida funcionó.

Conclusión práctica para la presentación: la demo en red se puede hacer con el hotspot de un celular. Para usar la red del colegio haría falta que alguien con permisos de administrador abra los puertos TCP 7777 y UDP 7778 en el firewall de cada PC (o que el colegio permita el tráfico entre equipos).

Qué quedó sin probar

- **Con 3 y con 5 jugadores.** Solo se probó con 2. El caso de 5 es el que más exige al servidor y a la red, así que sigue pendiente.
- **La red del colegio.** No se pudo comprobar en ninguna configuración.
- **Los números de la prueba.** No se midieron latencia ni consumo de red: el resultado "sin lag ni tirones" es una observación al jugar, no una medición.

Cómo repetir la prueba

1. Generar el ejecutable y copiarlo a cada PC (necesita Java 17 o superior, salvo que el ejecutable lo incluya).
2. Conectar todas las PC a la misma red (router o hotspot de un celular).
3. En la PC del host, `ipconfig` y anotar la dirección IPv4 de la placa conectada a esa red.
4. El host crea la partida; los demás se unen escribiendo esa IP.
5. Para probar pérdida de paquetes, poner `PERDIDA_UDP_SIMULADA` en 0,1 y volver a generar el ejecutable, o usar la tecla F8 en carrera si `DEBUG_RED` está activo (0 %, 20 %, 50 % y 100 %).
6. Si el invitado no puede unirse, revisar primero el firewall del host (TCP 7777 y UDP 7778) y que las dos PC estén en la misma red y no aisladas entre sí.

Commits

- Los commits de la etapa están en la rama `docs/pruebas-lan`. No hay cambios de código.

Etapa 10: Circuitos vectoriales y HUD estilo Fórmula 1

Objetivo

Cambiar por completo la estética de los circuitos: que ya no se construyan con cuadrados de pixel art (tiles) sino con líneas, de modo que las diagonales sean diagonales y los bordes no se vean escalonados. En el mismo trabajo se ajustó la física del auto, se hizo que la prueba local permita elegir el circuito y se reemplazó el HUD por una torre de tiempos como la de las transmisiones de Fórmula 1.

Esta etapa contradice una decisión del `CLAUDE.md` (los circuitos eran mapas de Tiled), por eso se avisó al grupo antes de empezar y el archivo se actualizó al cerrar.

Cómo se decidió

Se presentaron tres caminos antes de escribir código:

Opción	Qué es	Por qué sí o no
A. Solo cambiar el dibujo	Dibujar con líneas pero dejar la colisión en cuadrados de 16 px	Descartada: el auto chocaría contra escalones que no se ven
B. Todo vectorial	Colisión por geometría y reescribir `Carrera`, servidor y formato	Descartada: toca el servidor y el cruce de checkpoints, demasiado riesgo
C. Híbrido	El circuito se guarda como geometría, se dibuja con líneas y al cargarlo se rasteriza una grilla fina (4 px) solo para la colisión	**Elegida:** `Circuito` conserva su API, así que `Auto`, `Carrera` y el servidor no cambian

Qué se hizo

Pieza	Qué hace
`docs/FORMATO_CIRCUITO.md`	Especifica el formato JSON: línea central cerrada con un punto cada 8 px, ancho de asfalto y de pasto en cada punto, checkpoints, largada y cruces
`tools/circuitos/generar_circuitos.py`	Genera los 24 circuitos desde los trazados SVG de f1-circuits-svg (Jules Roy, CC BY 4.0); decide los anchos, separa los tramos pegados y arma checkpoints y largada
`CargadorCircuito` y `Circuito`	Leen el JSON, validan los campos y rasterizan la grilla de colisión (`Config.CELDA_COLISION`, 4 px); la API pública no cambió

Pieza	Qué hace
`VistaCircuito`	Arma una sola vez círculos apilados sobre cada punto de la línea central, en tres capas (muro, pasto y asfalto), con la misma forma que usa la colisión, y dibuja solo los bloques que entran en la cámara
Lanzador	Antialiasing de 4 muestras (`Config.MUESTRAS_ANTIALIASING`) para que los bordes diagonales salgan lisos
`Auto` y `ParametrosAuto`	Física más exigente (ver abajo)
`PruebaLocal`	Pantalla para elegir el circuito entre los 24 antes de correr; ESC en la carrera vuelve ahí con el mismo circuito
`TorreHud`	Torre de tiempos: una fila por auto con la posición (el puntero en rojo), la franja de su escudería, las tres letras del piloto y su mejor vuelta, más "35 RACING" y la vuelta del puntero
`tools/circuitos/verificacion` y la skill `circuitos`	Programas para comprobar los circuitos y las instrucciones para volver a generarlos



Chicane de Monza con la vista nueva



Mónaco, circuito callejero: asfalto y muro, sin pasto

Decisiones

- **El asfalto manda.** En cada punto de la pista el asfalto mide lo más ancho que permita el circuito (128 px como máximo; un 20 % menos en los de 3,4 km o menos, un poco menos en los intermedios) y solo se achica si otro tramo pasa tan cerca que no entra ni el muro, pero nunca baja de 72 px. El pasto es lo que sobra entre el asfalto y el muro, hasta 24 px de cada lado; donde no sobra nada el muro queda pegado al asfalto, como en Mónaco.
- **Callejeros sin pasto.** Los 9 circuitos callejeros (Albert Park, Jeddah, Miami, Gilles-Villeneuve, Mónaco, Madring, Bakú, Marina Bay y Las Vegas) son asfalto y muro; donde estaría el pasto hay asfalto, y se achicaron al 85 %. Los otros 15 conservan el pasto.
- **El muro es sólido hacia afuera.** El muro se dibuja como una línea de 6 px, pero para la colisión todo lo que está más allá del pasto es sólido: a la velocidad máxima el auto avanza unos 7,5 px por cuadro y atravesaría una línea tan fina.
- **Largo de vuelta calibrado con un bot.** El largo de cada circuito sale de una duración por una velocidad media. Para fijarla se escribió un bot que sigue la línea central con la física real; con `SEGUNDOS_VUELTA = 64` da 60,9 s de promedio en los 24.
- **Física.** Velocidad máxima de 450 px/s (antes 360), con la aceleración y los rozamientos escalados. La aceleración arranca en 300 px/s² y pierde el 70 % al llegar a la velocidad máxima, así que los últimos px/s cuestan. El radio mínimo de giro crece con el cuadrado de la velocidad (radio = velocidad² / 520), de modo que la línea de carrera importa más que acelerar a fondo, y girar cuesta un poco de velocidad. El zoom de la cámara pasa de 2 a 1,25. Estos valores se ajustaron en varias rondas según lo que se veía al jugar la prueba local.
- **Cargar el circuito sigue en el hilo de render.** Ya no se crean texturas, pero el archivo se lee con `Gdx.files` y la regla del proyecto es no tocar LibGDX desde los hilos de red. Se agregó una nota en `docs/PROTOCOLO.md`.
- **HUD sin diferencia con el líder ni neumáticos.** Para mostrarlos haría falta medir tiempos por checkpoint y mandarlos por red, o sea tocar el protocolo. La torre muestra la mejor vuelta. Tampoco se usó el logo de Fórmula 1, que es una marca registrada.

Problemas encontrados y cómo se resolvieron

Problema	Cómo se resolvió
En Miami, dos tramos quedaban tan pegados que un auto podía cortar camino entre ellos	Una prueba de atajos lo detectó (el camino más corto era el 31 % del recorrido por la pista); se subió el ancho mínimo y se separaron los tramos pegados. Hoy solo Suzuka da atajos, por su cruce
Las esquinas salían "mordidas" y los tramos cercanos "aplastados"	El dibujo provisorio armaba cada tramo como un cuadrilátero y dejaba muescas en las esquinas. Se lo reemplazó por la malla de círculos, que dibuja exactamente lo que choca
En Mónaco, una horquilla salía como una mancha	Las dos patas quedaban más cerca que el ancho de la pista. El generador ahora separa los tramos que quedan a menos de "ancho mínimo + 12 px" entre ejes (se movió el trazado 4 px como máximo)
Cuñas de pasto en las esquinas de los callejeros	Se probó abrir las esquinas del trazado limitando la curvatura, pero empeoró el dibujo de Mónaco y se descartó. Lo que se había pedido era que los callejeros no tuvieran pasto, y con eso las cuñas desaparecen
Las capturas de la ventana oculta salían dadas vuelta	El framebuffer de OpenGL tiene el origen abajo; el programa de captura invierte las filas antes de guardar



HUD en una carrera de prueba local, en Mónaco



Torre de tiempos con cinco autos de ejemplo: puntero, vuelta rápida, sin tiempo, terminado y abandonó

Verificación

Pruebas descartables con las clases reales del juego (`tools/circuitos/verificacion`) y capturas de una ventana oculta con la vista real:

Prueba	Resultado
Carga de los 24 circuitos	Todos cargan, entre 15 y 70 ms cada uno
Largada y checkpoints	Los 5 lugares de largada, toda la línea central y los centros de los checkpoints caen sobre pista y sin tocar muro, en los 24
Atajos (auto de radio 10 sobre la grilla de colisión)	23 circuitos sin atajos (ningún punto se alcanza con menos del 60 % del recorrido por la pista); Suzuka, por su cruce, llega a 0,34
Vuelta de un bot con la física real	Los 24 terminan; promedio 60,9 s
Capturas	Chicane de Monza, Mónaco, HUD en una carrera real y torre con cinco autos

El proyecto compila (`gradlew compileJava`). La prueba local con 2 jugadores se probó en el equipo de desarrollo.

Qué quedó sin hacer o sin probar

- **Pianos** en las curvas y el **puente** de Suzuka (el JSON ya guarda dónde cruza, pero el dibujo no lo usa).
- **Carrera en red con los circuitos nuevos:** el cambio no toca el protocolo ni el servidor, pero no se repitió la prueba en LAN de la etapa 9 con estos circuitos.
- **Los sprites** de los autos siguen siendo los provisorios.
- **Nueve circuitos callejeros** sin pasto: los autos chocan apenas salen del asfalto, así que se sienten más exigentes. Si resulta demasiado, se puede dar un margen.

Uso de IA, commits y pull request

El trabajo se hizo con Claude Code a partir de pedidos del grupo, con planes aprobados antes de cada cambio. Los commits salen de la cuenta de Joaquín Barreira y no llevan la línea `Co-Authored-By`; el uso de IA se declara en el README de esta carpeta. Rama de trabajo: `feature/estetica-f1`.

- `a25a455`: Feat: Circuitos vectoriales en JSON y generador desde SVG.
- `a9eb546`: Feat: Carga, colisión y dibujo de circuitos vectoriales, prueba local con elección y torre de tiempos.
- `025b627`: Feat: Física del auto más exigente.
- `a0985f0`: Refactor: Eliminar los mapas de Tiled y sus tilesets.
- `17441e5`: Docs: Etapa 10, skill de circuitos y verificación.

El pull request a `develop` todavía no se abrió.